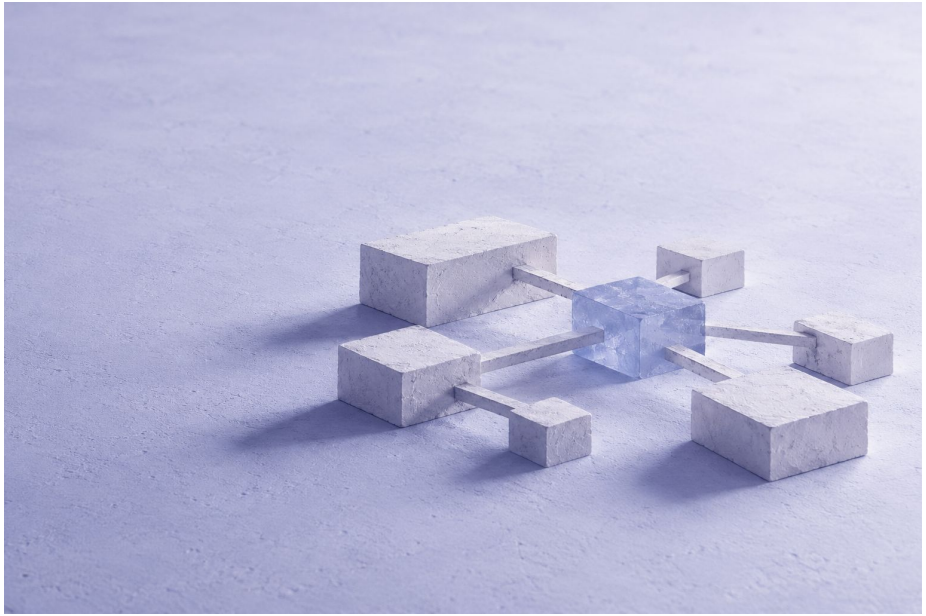
Чеклисты для агентского код-ревью

Составление, применение и поддержка актуальности чеклистов при агентском ревью изменений в backend API

prepared by: Darya Tolmeneva





Background

Код-ревью

Код-ревью — проверка исходного кода или изменений в нём другими разработчиками для обнаружения дефектов и улучшения качества.

Назначение:

- обнаружение ошибок;
- улучшение понятности и сопровождаемости кода;
- обмен знаниями внутри команды.

Примеры форматов:

- Формальная инспекция (Fagan)
- Совместный разбор (walkthrough)
- Современное код-ревью (modern code review)



Gonçalves et al. 2022, Empirical Software Engineering · Sadowski et al. 2018, ICSE-SEIP
Fagan 1976, IBM Systems Journal · Cohen 2009, Better Software

Стратегии проверки кода

01 Ad-hoc

Ревьюер сам выбирает, что и в каком порядке проверять, опираясь на собственный опыт.

02 Checklist-based

Заранее подготовленный список вопросов направляет внимание ревьюера на определённые аспекты кода.

03 Scenario-based

Сценарий задаёт порядок действий: что изучать, как выполнять проверку и на какие вопросы отвечать

04 Perspective-based reading

Вариант сценарного чтения. Проверка с позиций разных заинтересованных сторон: пользователя, тестировщика, проектировщика.

Gonçalves et al. 2022, Empirical Software Engineering · Basili et al. 1996, Empirical Software Engineering

Агентское код-ревью

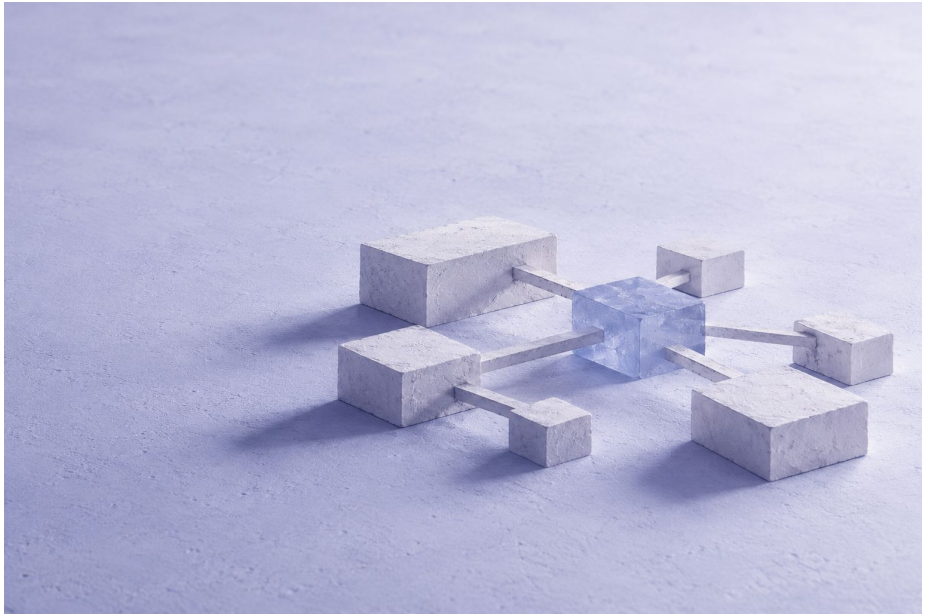
ИИ-агент в понимании Anthropic

“Agents are systems where LLMs dynamically direct their own processes and tool usage, maintaining control over how they accomplish tasks.”



Anthropic, 2024, *Building effective agents*





Проблемы

Которые я бы хотела решить

Составление чеклистов

Как отдельные пункты чеклиста влияют на качество ИИ-ревью?

425 вопросов признаны неподходящими

При анализе 1791 вопроса из студенческих чеклистов авторы выявили 425 неподходящих вопросов, в том числе неясных и не соответствующих заданию.

Chong et al. 2021

Подбор релевантных пунктов ещё не подтверждает пользу чеклиста

В эксперименте с разработчиками общий чеклист из 22 пунктов и адаптированный вариант из 7 не дали статистически значимого прироста обнаружения уязвимостей относительно одной инструкции искать проблемы безопасности.

Braz et al. 2022

Chong et al. 2021, ICSE-SEET · Braz et al. 2022, ICSE

Применение чеклистов

Как способ выполнения чеклиста и предоставляемый контекст влияют на качество и затраты ИИ-ревью?

Пошаговая подача не оказалась универсально лучше

Разработчики использовали одинаковые 18 пунктов: обычным списком или в пошаговом интерфейсе с привязкой к участкам кода. Устойчивого преимущества пошагового варианта на трёх заданиях не обнаружили — результаты зависели от задания.

Gonçalves et al. 2022

Разбиение сложной проверки повлияло на результат LLM

В iCodeReviewer сложную проверку раскладывают на простые утверждения. Модель оценивает их, а система объединяет ответы в решение. При отключении этого механизма F1 обнаружения уязвимостей снизилась с 63,98% до 60,24%.

Peng et al. 2025

Gonçalves et al. 2022, Empirical Software Engineering · Peng et al. 2025, ASE

Поддержка чеклистов

Как обновление чеклиста по результатам ревью влияет на качество последующих проверок?

Результаты ревью дают материал для обновлений

Авторы рекомендуют связывать найденные дефекты с пунктами чеклиста. Повторяющиеся проблемы, которые чеклист не покрывает, — разбирать и превращать в новые проверки.

Cohen et al. 2006

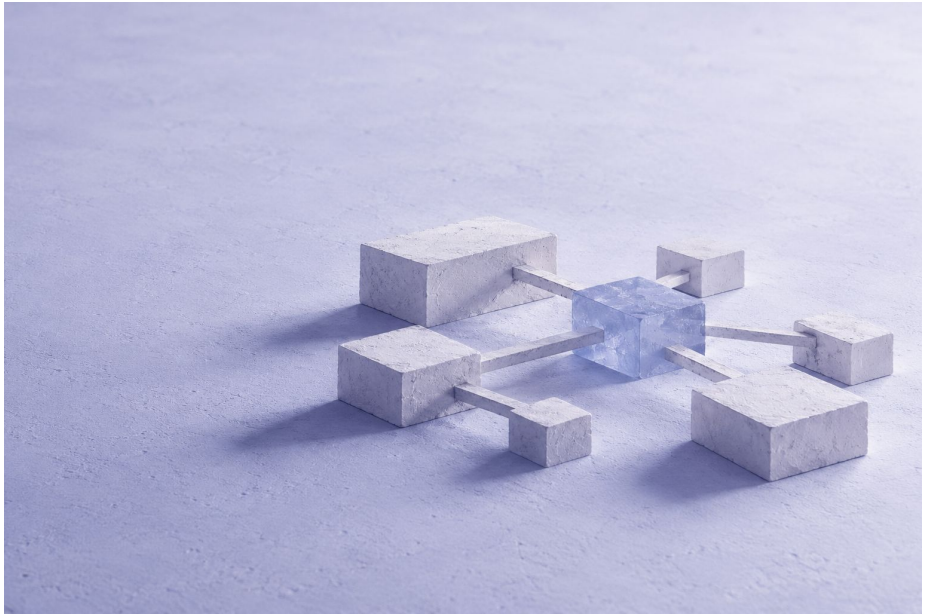
Накопленные правила требуют пересмотра

Принятые замечания ревью используют как материал для правил: человек вручную выделяет из них обобщаемые ошибки. Авторы предусматривают уточнение слишком широких правил, проверку дублирования и разрешение конфликтов между ними

Aggarwal & Ghalaty 2026

Cohen et al. 2006, Best Kept Secrets of Peer Code Review · Aggarwal & Ghalaty 2026, arXiv

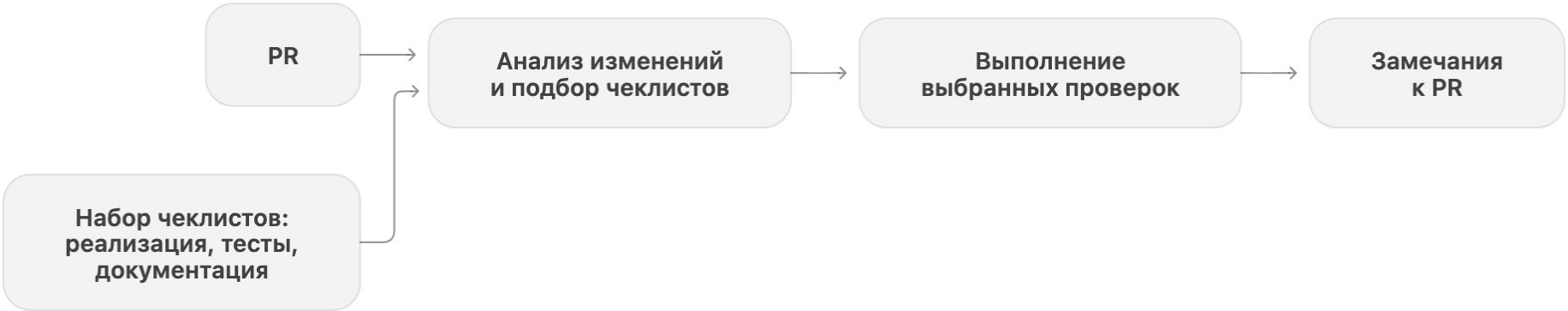




Подходы к решению

Предлагаемый подход

Система агентского ревью изменений в сфере backend API, которая автоматически подбирает подходящие чеклисты для проверки реализации, тестов и документации. Начальные чеклисты составляются вручную; способы их выполнения и полезность последующих обновлений исследуются экспериментально.



Ожидаемый вклад: система агентского код-ревью изменений backend API с автоматическим подбором тематических чеклистов и экспериментально обоснованные рекомендации по их составу, способам выполнения и обновлению с учётом качества и времени ревью

Существующие подходы

Система	Как используются критерии проверки	Отличие предлагаемого подхода
LLM-Reviewer	**Критерии**: фиксированный чеклист — 14 проверок в 8 категориях. Например: именование, комментарии, дублирование, инициализация переменных и логические ошибки. **Применение**: весь чеклист передаётся модели вместе с изменениями; ответ формируется по категориям.	Автоматический подбор отдельных чеклистов реализации, тестов и документации API. Экспериментальная оценка вклада их пунктов и блоков.
iCodeReviewer	**Критерии**: 38 специализированных цепочек промптов для поиска уязвимостей. **Применение**: проверки выбираются по AST-признакам; сложные условия анализируются по шагам, при необходимости извлекается дополнительный контекст.	Тематические проверки реализации, тестов и документации backend API. Сравнение способов выполнения одинакового набора проверок и предоставления контекста.
RovoDev — Atlassian	**Критерии**: GCode — проверки кода; GTest — покрытие и качество тестов; GComment — оформление замечаний. **Применение**: три блока включаются в единый промпт с diff, описанием PR и Jira; отдельный выбор блоков не описан.	Автоматический подбор тематических чеклистов, включая документацию API. Оценка вклада отдельных пунктов и блоков, тогда как авторы удаляли рекомендации совместно.
AutoCommenter — Google	**Критерии**: практики программирования из руководств Google. **Применение**: по исходному файлу модель предсказывает место нарушения и ссылку на правило; результаты фильтруются, малоценные правила подавляются по обратной связи.	Выполнение явного набора проверок на уровне PR. Контролируемое сравнение исходных и обновлённых чеклистов на одинаковых новых изменениях.

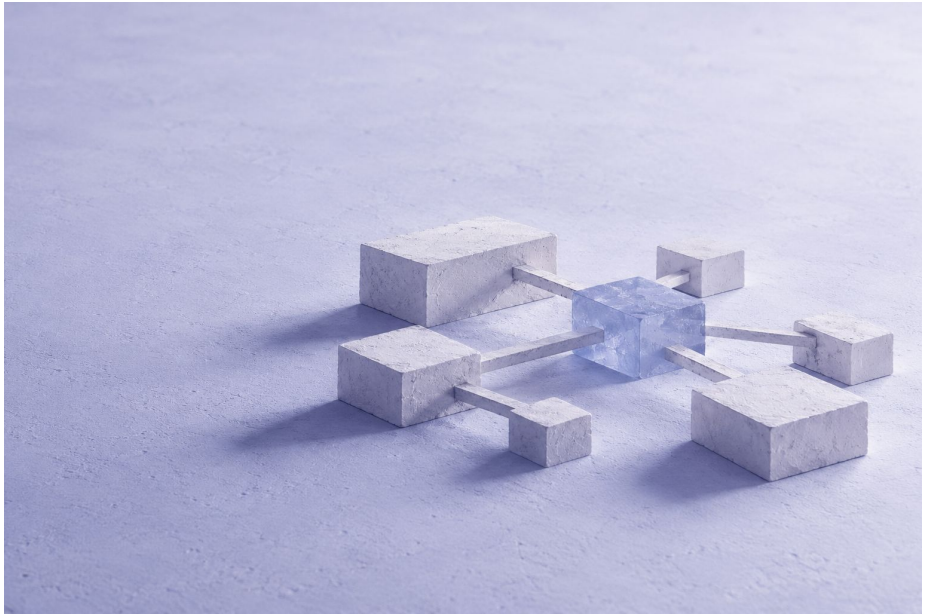
Oliveira et al. 2026, ICSE-SEET · Peng et al. 2025, ASE
Tantithamthavorn et al. 2026, ICSE-SEIP · Vijayvergiya et al. 2024, AIware

Существующие подходы

Система	Как используются критерии проверки	Отличие предлагаемого подхода
SWE-Review	**Критерии**: соответствие исправления исходной задаче и отсутствие регрессий. **Применение**: агент изучает причину ошибки, формулирует ожидаемое исправление, анализирует патч и запускает проверки по заданному протоколу.	Тематические чеклисты реализации, тестов и документации API. Раздельная оценка их состава и способов выполнения при фиксированных остальных условиях.
RepoReviewer	**Критерии**: конкретные списки проверок в статье не приведены. **Применение**: после сбора контекста модель анализирует выбранные файлы и формирует находки с местом, серьёзностью, описанием и предложением исправления.	Набор чеклистов для реализации, тестов и документации API с автоматическим подбором под PR. Экспериментальная оценка пользы отдельных пунктов и обновлений.
Qodo PR-Agent / Beko	**Критерии**: статья не описывает, по каким правилам и промптам доработанный PR-Agent формирует замечания. **Применение**: доработанный PR-Agent автоматически запускается при создании PR в Azure DevOps и публикует замечания.	Фиксация конкретных критериев и их версий. Контролируемое сравнение вариантов чеклистов и выполнения; работа Beko оценивает преимущественно эффект промышленного внедрения.

Wang et al. 2026, arXiv · Zhang 2026, arXiv · Cihan et al. 2025, ICSE-SEIP





Проверка результата

Датасет

Пример	Как устроены данные
SWE-Review-Bench	1384 патча для 500 задач: корректные и ошибочные решения, репозитории и исполняемые тесты.
Jin & Chen	Парные реализации одного требования: корректная и дефектная. Позволяют проверять обнаружение ошибок и необоснованное отклонение правильного кода.
Gonçalves et al.	Три подготовленных изменения кода текстового редактора jEdit, написанного на Java. Содержат 3, 9 и 10 известных функциональных дефектов — исходных и специально внедрённых.
Braz et al., Less is More	Специально подготовленное Java-изменение с двумя уязвимостями. Материалы проверили эксперты и доработали в пилотных запусках. Пример создания контролируемых заданий.
iCodeReviewer	682 примера: 345 с подтверждёнными проблемами безопасности и 337 с подтверждёнными ложными срабатываниями. Внутренний набор; полезен как пример состава и разметки данных.

Wang et al. 2026, arXiv · Jin & Chen 2026, Automated Software Engineering
Gonçalves et al. 2022, Empirical Software Engineering · Braz et al. 2022, ICSE · Peng et al. 2025, ASE

План подготовки данных

- Отобрать примеры изменений реализации, тестов и документации backend API из открытых наборов.
- Сохранить контекст каждого примера: репозиторий, исходный коммит, описание задачи и diff изменения.
- При необходимости искусственно включить корректные и дефектные варианты по выбранным критериям проверки.
- Для каждого PR зафиксировать известные проблемы, основания для их подтверждения и применимые чеклисты.

Эксперимент

Что исследуем	Что сравниваем
Состав чеклистов	Без чеклистов ↔ полный применимый набор ↔ тот же набор без выбранных пунктов или блоков.
Выполнение и контекст	Чеклист целиком ↔ пошаговое выполнение при одинаковом контексте. Отдельно: общий контекст для всех проверок ↔ контекст, подобранный под отдельные проверки.
Обновление чеклистов	Исходные чеклисты ↔ чеклисты, доработанные агентом на отдельном наборе PR. Оба варианта фиксируем и сравниваем на новых PR итогового набора.

Условия сравнения

- Одинаковые PR, модель, настройки и общие инструкции; несколько запусков каждого варианта.
- Выбранные системой чеклисты сверяем с заранее определённым применимым набором.
- Удаляем один пункт или блок чеклиста и повторяем ревью того же PR. Остальные проверки и условия оставляем одинаковыми.

Оценка помощи ревьюеру

Сравниваем самостоятельное ревью и ревью с результатами системы на сопоставимых PR. Измеряем качество итогового ревью и время работы человека. После каждого задания участники заполняют StuMMBE-Q.

Метрики

Метрика	Что измеряем
Precision — точность	Доля подтверждённых проблем среди всех уникальных проблем, указанных в результате ревью.
Recall — полнота	Доля обнаруженных известных проблем среди всех известных проблем в наборе.
F1	Совместная оценка точности и полноты: 2 × Precision × Recall / (Precision + Recall).
Время ревью	Отдельно: длительность работы системы и время работы человека над ревью.
Расход токенов	Суммарный расход по всем вызовам модели на одно ревью.
Устойчивость результатов	Разброс качества, длительности работы системы и расхода токенов при повторных запусках на одинаковых PR.
Умственная нагрузка — StuMMBE-Q, Mental Load	Насколько сложным человек воспринимает выполнение ревью. Среднее по 6 вопросам со шкалой от 1 до 7.
Умственные усилия — StuMMBE-Q, Mental Effort	Сколько умственных усилий человек вкладывает в ревью. Среднее по другим 6 вопросам со шкалой от 1 до 7.

Gonçalves et al. 2022, Empirical Software Engineering · Krell 2017, Cogent Education

Метрики

Категория	Критерий
Instrumental	Известный дефект обнаружен; тип и расположение указаны верно, описание понятно объясняет проблему.
Helpful	Ответ помогает обнаружить известный дефект, хотя тип, расположение или описание частично неточны.
Misleading	Ответ не помогает обнаружить известный дефект. Например, система сообщает об отсутствии проблем при наличии дефекта.
Uncertain	Известный дефект пропущен, но указана другая возможная проблема, которую оценщики пока не могут подтвердить или опровергнуть.

Показатель	Формула
I-Score — точные обнаружения	I / N × 100%
IH-Score — точные и частично полезные результаты	(I + H) / N × 100%
M-Score — результаты без полезной информации об известном дефекте	M / N × 100%
Доля результатов категории Uncertain	U / N × 100%

Yu et al. 2026, arXiv

Источники

1. Aggarwal, A., & Ghalaty, N. F. (2026). *Self-Improving AI Coding Agents Through Accumulated Behavioral Rules: A Closed-Loop Framework*. arXiv:2607.13091.
2. Anthropic. (2024). *Building Effective Agents*. Anthropic Engineering.
3. Basili, V. R., et al. (1996). *The Empirical Investigation of Perspective-Based Reading*. Empirical Software Engineering, 1(2), 133–164.
4. Braz, L., et al. (2022). *Less is More: Supporting Developers in Vulnerability Detection during Code Review*. ICSE 2022, 1317–1329.
5. Chong, C. Y., Thongtanunam, P., & Tantithamthavorn, C. (2021). *Assessing the Students' Understanding and their Mistakes in Code Review Checklists: An Experience Report of 1,791 Code Review Checklist Questions from 394 Students*. ICSE-SEET 2021, 20–29.
6. Cihan, U., et al. (2025). *Automated Code Review in Practice*. ICSE-SEIP 2025, 425–436.
7. Cohen, J., Teleki, S., & Brown, E. (2006). *Best Kept Secrets of Peer Code Review*. SmartBear Software. Использована глава "Questions for a Review Process", 103–114.
8. Cohen, J. (2009). *Prepare to Succeed: A Guide to Effective Code Review*. Better Software, November/December, 30–35.
9. Fagan, M. E. (1976). *Design and Code Inspections to Reduce Errors in Program Development*. IBM Systems Journal, 15(3), 182–211.
10. Gonçalves, P. W., et al. (2022). *Do Explicit Review Strategies Improve Code Review Performance? Towards Understanding the Role of Cognitive Load*. Empirical Software Engineering, 27, article 99.
11. Jin, H., & Chen, H. (2026). *Are LLMs Reliable Code Reviewers? Systematic Overcorrection in Requirement Conformance Judgement*. Automated Software Engineering, 33, article 90.
12. Krell, M. (2017). *Evaluating an Instrument to Measure Mental Load and Mental Effort Considering Different Sources of Validity Evidence*. Cogent Education, 4(1), article 1280256.
13. Oliveira, E., et al. (2026). *AI-Assisted Code Review as a Scaffold for Code Quality and Self-Regulated Learning: An Experience Report*. ICSE-SEET 2026.
14. Peng, Y., et al. (2025). *iCodeReviewer: Improving Secure Code Review with Mixture of Prompts*. ASE 2025.
15. Sadowski, C., et al. (2018). *Modern Code Review: A Case Study at Google*. ICSE-SEIP 2018, 181–190.
16. Tantithamthavorn, K., et al. (2026). *RovoDev Code Reviewer: A Large-Scale Online Evaluation of LLM-based Code Review Automation at Atlassian*. ICSE-SEIP 2026.
17. Vijayvergiya, M., et al. (2024). *AI-Assisted Assessment of Coding Practices in Modern Code Review*. AIware 2024.
18. Wang, R., et al. (2026). *SWE-Review: Closing the Loop on Issue Resolution with Agentic Code Review*. arXiv:2607.06065.
19. Yu, J., et al. (2026, версия v6). *An Insight into Security Code Review with LLMs: Capabilities, Obstacles, and Influential Factors*. arXiv:2401.16310v6. Первая версия опубликована в 2024 году; используемая редакция — от 9 мая 2026 года.
20. Zhang, P. (2026). *RepoReviewer: A Local-First Multi-Agent Architecture for Repository-Level Code Review*. arXiv:2603.16107.

Спасибо за внимание